A Mini Project Report of

PCC-201-COM : Data Structures

Submitted by

Kokil Vakratund Prakash (75)

Under the Guidance of

Prof. Tejal. H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]



BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Kokil Vakratund Prakash (75),** on **"Implementation of a basic version of a web browser's back button functionality [Stack]", this work is done and submitted during 2026–2**027 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. Tejal. H. Patil
(Guide)

Dr. Lalit. V. Patil
(Head of Computer Engg. Department)

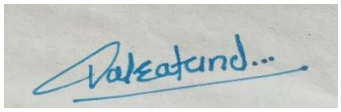
ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal. H. Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit. V. Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Kokil Vakratund Prakash
Student Name



ABSTRACT

The back button is an essential feature of web browsers that allows users to navigate to previously visited web pages. This project focuses on implementing a basic version of a web browser's back button functionality using a stack data structure. A stack follows the Last In, First Out (LIFO) principle, making it suitable for storing the browsing history of a user.

Whenever a user visits a new web page, the current page is pushed onto the stack. When the user clicks the back button, the most recently visited page is popped from the stack, allowing the user to return to the previous page. This implementation demonstrates the practical application of stack operations such as push() and pop() in managing navigation history.

The proposed system provides a simple and efficient way to simulate browser back navigation while helping users understand the role of stack data structures in real-world applications. The project highlights the importance of stacks in managing sequential operations and serves as a foundation for understanding more advanced browser history mechanisms.

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.

Acknowledgement i
Abstract ii
Content iii
List Of Figures iv

1 Introduction 1
   a) Overview
   b) Problem Statement
   c) Objective

2 Literature Survey 2

3 UML Diagram 3

4 System Requirement 4

5 System Design 5
   a) Existing system
   b) Proposed system
   c) Working Diagram
   d) System code

6 Result 8

7 Application 9

8 Future Scope 10

9 Conclusion 11

10 References 12

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Case Diagram	03
Fig 5.1	System Design	04
Fig 5.2	System Code	07
Fig 5.3	Result	09

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

INTRODUCTION

1.1 Overview

A web browser is a software application that allows users to access and navigate websites on the Internet. During browsing, users often visit multiple web pages and may need to return to previously visited pages. The Back button is an important browser feature that enables users to navigate to an earlier page without manually entering its address again. This mini-project focuses on implementing a basic version of this functionality using the Stack data structure.

A stack is a linear data structure that follows the Last In, First Out (LIFO) principle, in which the last element inserted is the first element removed. In this project, the stack is used to manage the history of visited web pages. Whenever a user visits a new page, its address or name is pushed onto the stack. When the user performs a Back operation, the current page is popped from the stack, allowing navigation to the previously visited page. The peek operation can also be used to view the top element without removing it.

The project demonstrates how a fundamental data structure can be applied to a real-world problem. It provides a simple understanding of browser navigation history and the use of stack operations in software development. The implementation is intended for educational purposes and represents only the basic Back button functionality rather than the complete history management system of a modern web browser.

1.2 Objectives

The main objective of this mini-project is to implement the basic Back button functionality of a web browser using the Stack data structure. The specific objectives are:

- To understand the concept and working principle of a stack and its LIFO behaviour.
- To implement stack operations such as push, pop, and peek for managing web page history.
- To simulate navigation between web pages and allow users to return to previously visited pages.
- To handle situations where no previous page is available in the stack.
- To demonstrate the practical application of data structures in solving real-world problems.

1. LITERATURE SURVEY

Various researchers have studied airline reservation systems to improve booking operations, information management, and customer convenience. Research on sorting algorithms has also explored methods for understanding and applying these algorithms effectively. The following studies provide a foundation for the development of the proposed Flight Reservation System using searching and sorting techniques.

Year	Paper Name	Author	Conclusion
2025	Application of Stack Data Structure in Application Development	Sarah Amaylia, Viktoria Angelita Setiabud, Renza Alvianino, Rahmat ugroho Saputra, Helena Kusuma Wardhani, and Azis Suroni.	The study highlights the usefulness of the stack data structure in application development and its practical applications in navigation and other operations.
2025	Advancing Web Browser Forensics: Critical Evaluation of Emerging Tools and Techniques	Rishal Ravikesh Chand,NeerajAnandSharma , and Muhammad Ashad Kabir.	The study shows that browser history and related data can be recovered and analyzed, emphasizing the importance of effective browser data management.
2025	IRCHIVER: An Information-Centric Personal Web Archive for Revisiting Past Online Sensemaking Tasks	Zhicheng Huang and Jeff Huang.	The study demonstrates that storing and searching previously viewed web content helps users revisit information and retrace their browsing activities.
2026	An Intelligent Browser History Forensics Method for Automated Analysis of Web Activity Logs, Credentials, and User Behavioral Profiles	Leila Rzayeva, Aliya Zhetpisbayeva, Alisher Batkuldin, Nursultan Nyssanov, Alissa Ryzhova, and Faisal Saeed.	The study demonstrates how automated techniques can analyze browser history to identify user activities and detect suspicious behaviour.

3. UML DIAGRAM

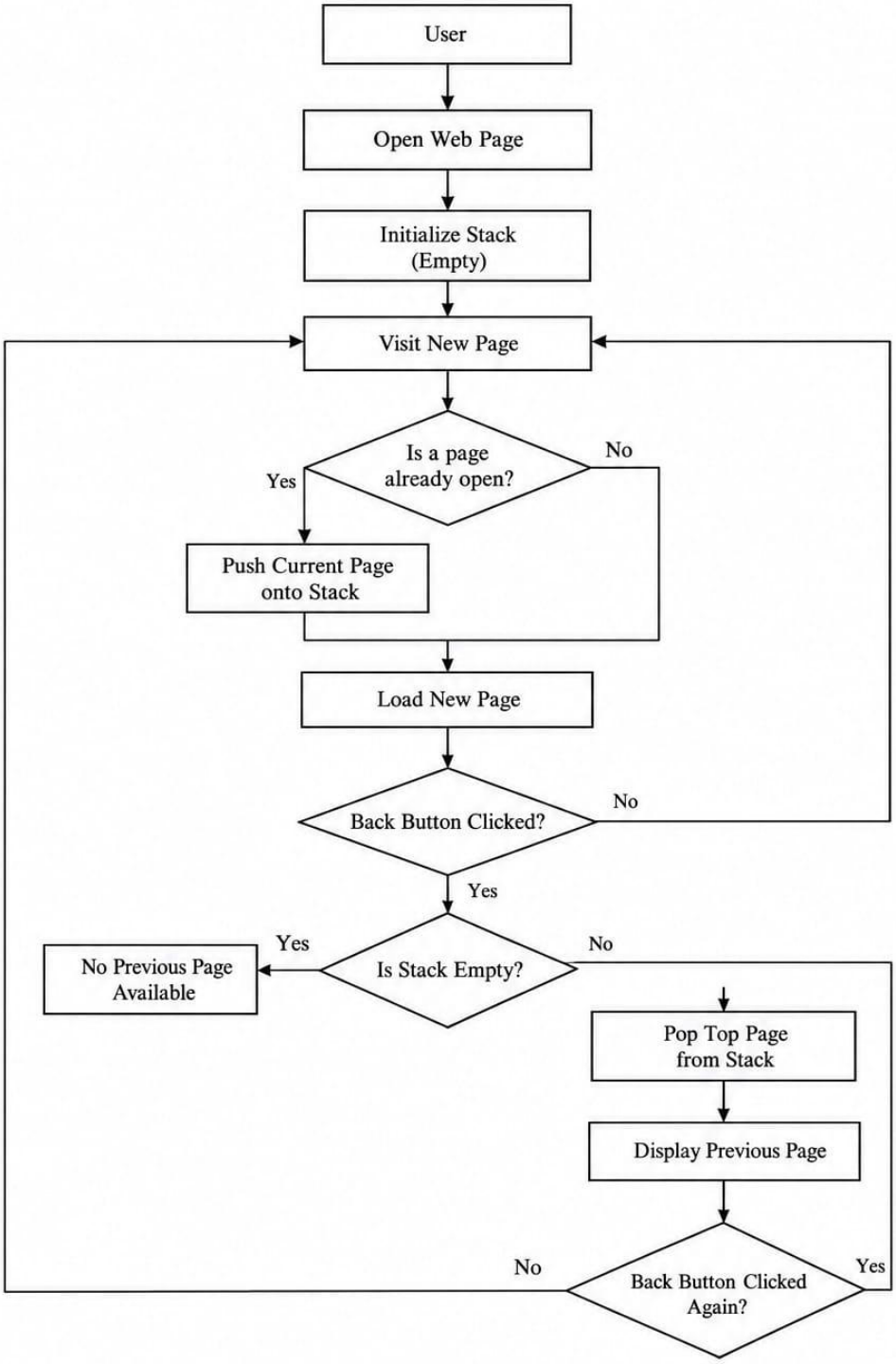


Fig. 3.1: Flow Diagram of Web Browser Back Button (Stack)

2. SYSTEM REQUIREMENT

2.1 Hardware Requirements

The hardware requirements for implementing the basic version of a web browser's Back button functionality using a Stack data structure are as follows:

- Processor: Intel Dual-Core processor or equivalent
- RAM: Minimum 2 GB
- Hard Disk: At least 100 MB of free storage space
- Input Devices: Keyboard and mouse
- Output Device: Monitor or display screen

2.2 Software Requirements

The software requirements for developing and executing the mini-project are as follows:

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: C, C++, Java, or Python, depending on the implementation
- Code Editor/IDE: Visual Studio Code, Code Blocks, Eclipse, or any suitable editor
- Compiler/Interpreter: A suitable compiler or interpreter for the selected programming language
- Data Structure: Stack (Last In, First Out – LIFO)
- Testing Environment: Terminal, command prompt, or IDE console

3. SYSTEM DESIGN

3.1 Existing System

In the existing system, web browsers provide a Back button that allows users to return to previously visited web pages. This functionality is generally managed by the browser's built-in history mechanism, which records the pages visited by the user. Modern browsers maintain navigation history and provide additional features such as Forward navigation, history lists, and session restoration.

However, the internal implementation of browser history management can be complex for beginners to understand. It involves managing page navigation, maintaining history records, and handling different browsing situations. Therefore, a simplified implementation is useful for understanding the basic logic behind the Back button using fundamental data structures.

4.2 Proposed System

The proposed system implements a basic version of a web browser’s Back button functionality using the Stack data structure. A stack follows the Last In, First Out (LIFO) principle, where the last element inserted is the first element removed.

In this system, previously visited pages are stored in a stack. When the user visits a new page, the current page is pushed onto the stack. When the Back button is clicked, the system pops the previous page from the stack and displays it. If the stack is empty, an appropriate message is displayed.

The main features of the proposed system are:

- Stack-Based History: Stores previously visited web pages.
- Back Navigation: Allows users to return to the previous page.
- Empty Stack Handling: Displays a message when no previous page is available.
- Simple Implementation: Uses basic stack operations such as push and pop.
- Future Enhancement: Can be extended to support Forward navigation.

Entity Relationship Diagram

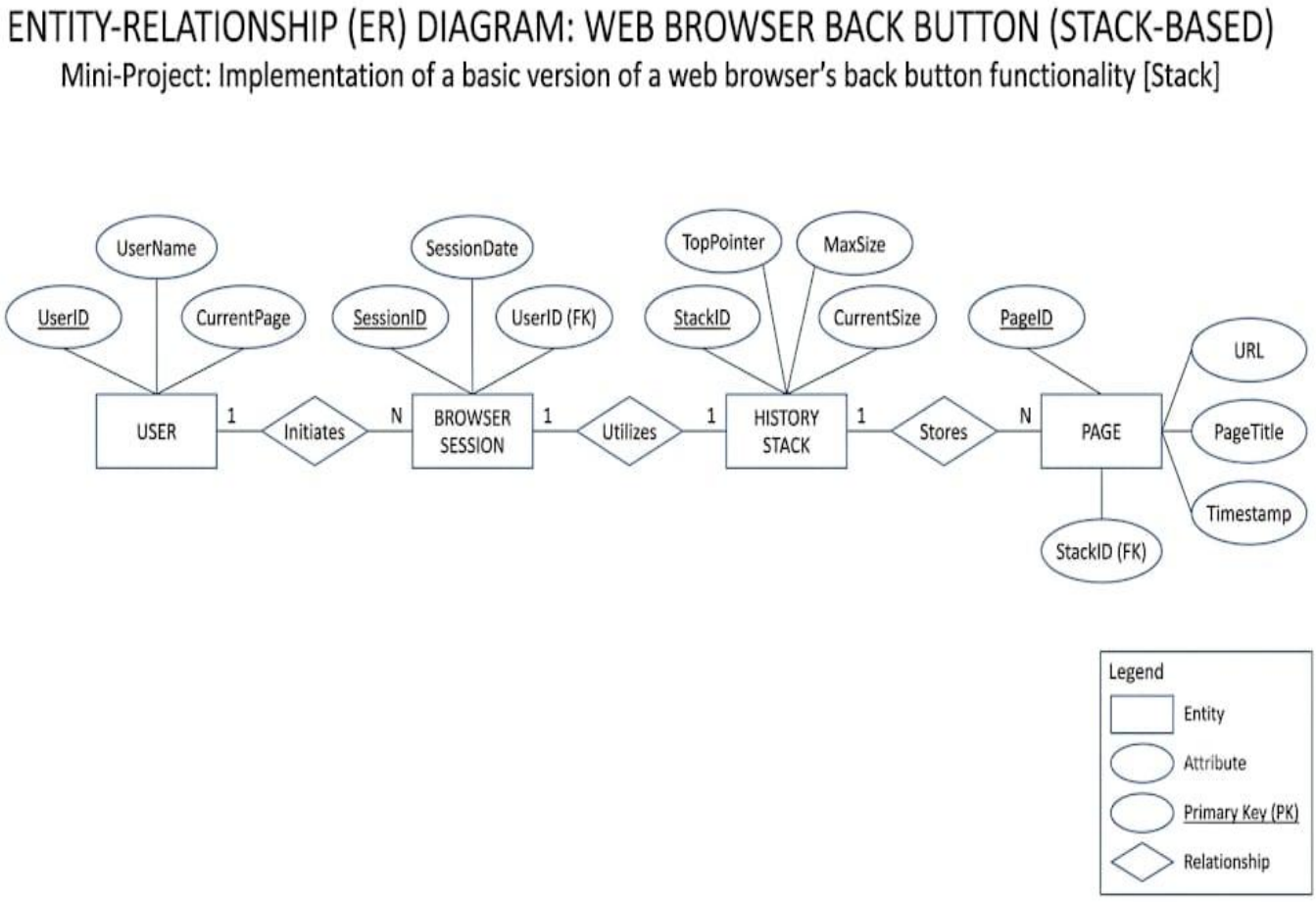


Fig. Entity relationship diagram of Browser's Back Button Functionality

SYSTEM CODE

In [ ]:
```
%pip install ipywidgets
```

In [7]:
```
##### # Mini Project: Web Browser Back Button Using Stack
# Features: Back, Stack Display, Clear History, History Limit

class Browser:
    def __init__(self, max_history=5):
        self.history = []
        self.current_page = None
        self.max_history = max_history

    # Visit a new webpage
    def visit_page(self, url):
        url = url.strip()

        if not url:
            print("Error: URL cannot be empty!")
            return

        if self.current_page is not None:
            self.history.append(self.current_page)

            # Remove the oldest page if the limit is exceeded
            if len(self.history) > self.max_history:
                removed = self.history.pop(0)
                print(f"Oldest history removed: {removed}")

        self.current_page = url
        print(f"Successfully visited: {url}")

    # Back button using stack POP
    def go_back(self):
        if not self.history:
            print("No previous page available!")
            return

        self.current_page = self.history.pop()
        print(f"Navigated back to: {self.current_page}")

    # Show current page
    def show_current_page(self):
        print("\nCurrent webpage:", self.current_page or "None")

    # Display stack contents
    def show_history(self):
        print("\n--- Browsing History Stack ---")

        if not self.history:
            print("Stack is empty.")
        else:
            for i, page in enumerate(self.history):
                marker = " <- TOP" if i == len(self.history) - 1 else ""
                print(f"{i + 1}. {page}{marker}")
```

```
        print(f"Stack size: {len(self.history)}/{self.max_history}")

    # Clear browsing history but keep current page
    def clear_history(self):
        self.history.clear()
        print("Browsing history cleared!")
        print("Current webpage remains unchanged.")

    # Main menu
    def run(self):
        while True:
            print("\n" + "=" * 38)
            print("       WEB BROWSER SIMULATOR")
            print("       Back Button Using Stack")
            print("=" * 38)
            print("1. Visit a new webpage")
            print("2. Back")
            print("3. Show current webpage")
            print("4. Display stack contents")
            print("5. Clear browsing history")
            print("6. Exit")

            choice = input("Enter your choice (1-6): ").strip()

            if choice == "1":
                url = input("Enter webpage URL: ")
                self.visit_page(url)

            elif choice == "2":
                self.go_back()

            elif choice == "3":
                self.show_current_page()

            elif choice == "4":
                self.show_history()

            elif choice == "5":
                self.clear_history()

            elif choice == "6":
                print("Browser simulation ended.")
                break

            else:
                print("Invalid choice! Enter a number from 1 to 6.")


# Run the browser
browser = Browser(max_history=5)
browser.run()
```

4. RESULT

```
=======================================
        WEB BROWSER SIMULATOR
        Back Button Using Stack
=======================================
1. Visit a new webpage
2. Back
3. Show current webpage
4. Display stack contents
5. Clear browsing history
6. Exit
Successfully visited: google.com

=======================================
        WEB BROWSER SIMULATOR
        Back Button Using Stack
=======================================
1. Visit a new webpage
2. Back
3. Show current webpage
4. Display stack contents
5. Clear browsing history
6. Exit
Current webpage: google.com

=======================================
        WEB BROWSER SIMULATOR
        Back Button Using Stack
=======================================
1. Visit a new webpage
2. Back
3. Show current webpage
4. Display stack contents
5. Clear browsing history
6. Exit
Successfully visited: vakratund.com

=======================================
        WEB BROWSER SIMULATOR
        Back Button Using Stack
=======================================
1. Visit a new webpage
2. Back
3. Show current webpage
4. Display stack contents
5. Clear browsing history
6. Exit
```

```
--- Browsing History Stack ---
1. google.com <- TOP
Stack size: 1/5

========================================
        WEB BROWSER SIMULATOR
        Back Button Using Stack
========================================
1. Visit a new webpage
2. Back
3. Show current webpage
4. Display stack contents
5. Clear browsing history
6. Exit
Browsing history cleared!
Current webpage remains unchanged.

========================================
        WEB BROWSER SIMULATOR
        Back Button Using Stack
========================================
1. Visit a new webpage
2. Back
3. Show current webpage
4. Display stack contents
5. Clear browsing history
6. Exit
Current webpage: vakratund.com

========================================
        WEB BROWSER SIMULATOR
        Back Button Using Stack
========================================
1. Visit a new webpage
2. Back
3. Show current webpage
4. Display stack contents
5. Clear browsing history
6. Exit
Browser simulation ended.
```

5. APPLICATIONS

The applications of implementing a basic version of a web browser's Back button functionality using a Stack data structure are:

1. **Web Browser Navigation:** The stack data structure allows users to return to previously visited web pages by storing page history and retrieving the most recent previous page.
2. **Browser History Management:** Stack operations help maintain browsing history and make it easier to manage the sequence of visited pages.
3. **Undo Operations:** The same stack principle is used in text editors and other applications to reverse recent actions and restore previous states.
4. **Mobile Application Navigation:** Stack-based navigation is used in mobile applications to help users return to previously opened screens or pages.
5. **Software Development:** This project helps developers understand how stack operations such as push and pop can be applied to real-world navigation problems.
6. **Educational Purposes:** The project provides practical knowledge of the Stack data structure and its Last In, First Out (LIFO) principle, helping students understand data structures through a simple application.

6. FUTURE WORK

The proposed system can be enhanced in the future by adding more features to improve its functionality and usability.

1. **Forward Button Functionality:** Implement a Forward button to return to pages visited after using the Back button.
2. **Complete Browser History:** Maintain a complete list of visited web pages with their URLs and titles.
3. **Graphical User Interface (GUI):** Develop a user-friendly interface to make navigation easier.
4. **History Management:** Allow users to view, search, and clear their browsing history.
5. **Multiple Tab Support:** Manage separate browsing histories for multiple tabs.
6. **Data Persistence:** Store browsing history so that it can be restored when the application is reopened.

These enhancements can make the system more practical and provide a foundation for developing a more advanced browser navigation system.

7. CONCLUSION

The mini-project, “Implementation of a Basic Version of a Web Browser’s Back Button Functionality Using Stack,” demonstrates the practical application of the Stack data structure in browser navigation. The system allows users to return to previously visited web pages using the Last In, First Out (LIFO) principle.

In this project, previously visited pages are stored in a stack. Whenever a user visits a new page, the current page is pushed onto the stack. When the Back button is clicked, the pop operation retrieves the previous page and displays it. If no previous page is available, the system displays an appropriate message.

This project provides a clear understanding of stack operations, including push and pop, and their use in solving real-world problems. It also improves programming and problem-solving skills. In the future, the system can be enhanced by adding Forward navigation, browsing history management, multiple-tab support, and a graphical user interface. Overall, the project demonstrates how a basic data structure can be used to implement browser navigation functionality.

8. REFERENCE

[1] Mozilla Developer Network (MDN). "History: back() Method." Available at: https://developer.mozilla.org/en-US/docs/Web/API/History/back

[2] Mozilla Developer Network (MDN). "Working with the History API." Available at: https://developer.mozilla.org/en-US/docs/Web/API/History_API/Working_with_the_History_API

[3] Oracle. "Class Stack<E> — Java Platform, Standard Edition Documentation." Available at: https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/util/Stack.html

[4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*, 4th Edition. The MIT Press. Available at: https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

[5] Mozilla Developer Network (MDN). "History API." Available at: https://developer.mozilla.org/en-US/docs/Web/API/History_API

[6] GeeksforGeeks. "Implementing Backward and Forward Buttons of Browser." Available at: https://www.geeksforgeeks.org/dsa/implementing-backward-and-forward-buttons-of-browser/